Local Voice AI Assistant – System Architecture

1. Project Overview

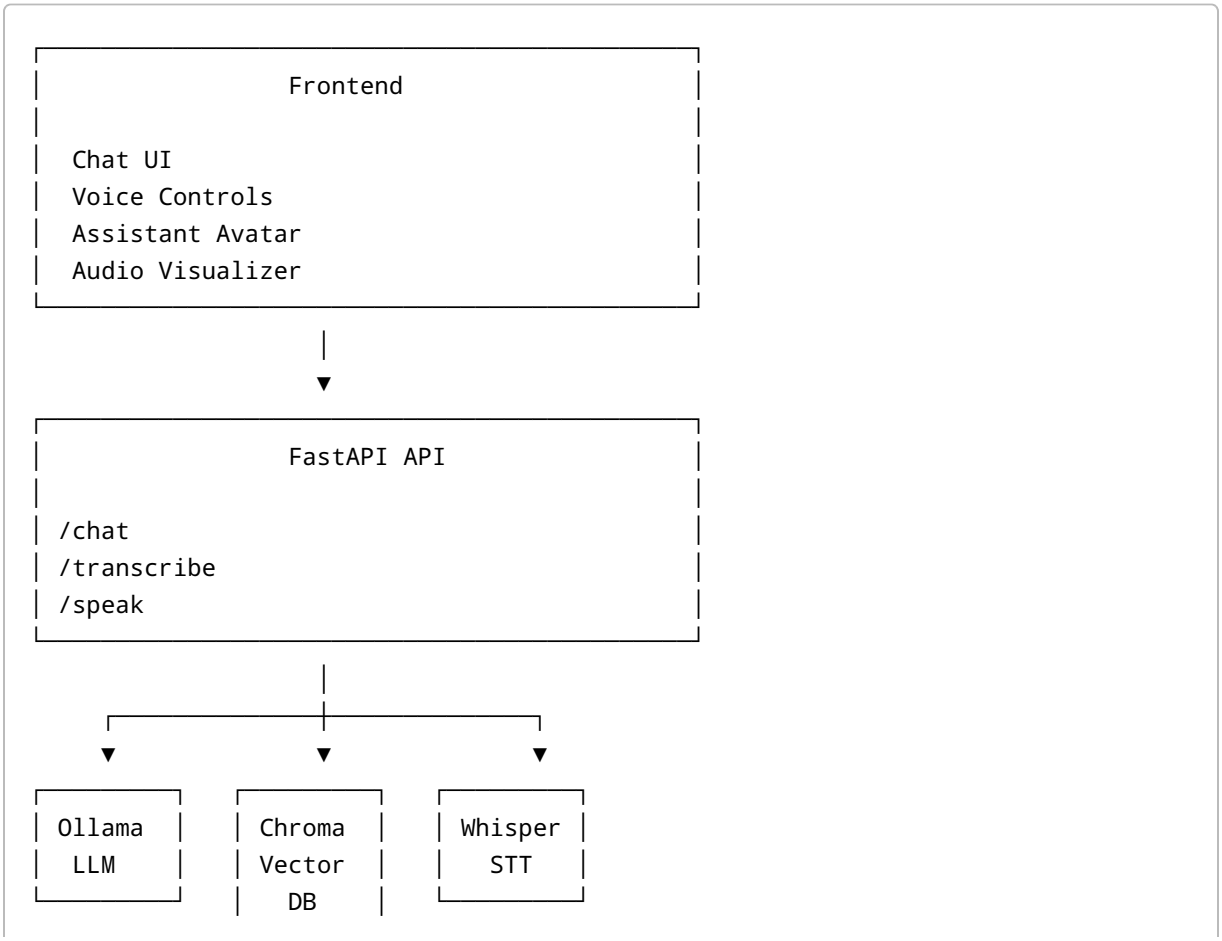
Local Voice AI Assistant is a fully self-hosted conversational AI platform that combines:

- Retrieval Augmented Generation (RAG)
- Local Large Language Models (LLMs)
- Speech-to-Text (STT)
- Text-to-Speech (TTS)
- Interactive Web Chat Interface
- Animated Assistant Avatar
- Real-Time Audio Visualization

The system is designed to operate entirely on local infrastructure without dependence on external AI APIs.

---

2. High-Level Architecture



```
└──────────┘
      │
      ▼
Knowledge Base
   (.txt)
      │
      ▼
 Embeddings
      │
      ▼
Retrieval Layer

      │
      ▼

Kokoro TTS
      │
      ▼

Audio Output
```

---

3. Backend Components

3.1 FastAPI Service

FastAPI serves as the orchestration layer.

Responsibilities:

- Receives user messages
- Accepts audio uploads
- Invokes RAG pipeline
- Invokes TTS pipeline
- Returns responses to frontend

Endpoints:

POST /chat

Input:

```
{
  "message": "What services do you offer?"
}
```

Output:

```
{
  "response": "We provide AI automation services."
}
```

POST /transcribe

Input:

Audio file upload

Output:

```
{
  "text": "What services do you offer?"
}
```

POST /speak

Input:

```
{
  "message": "Hello, how can I help?"
}
```

Output:

Generated WAV audio stream

---

4. Retrieval Augmented Generation Layer

Objective

Provide grounded responses based on local company knowledge.

Knowledge Source

Current implementation:

```
knowledge/
└── knowledge.txt
```

Possible future sources:

- PDFs
- Word Documents
- Websites
- Databases
- CRM Systems

Ingestion Pipeline

Step 1

Read source document.

Step 2

Split content into chunks.

Example:

```
Chunk 1
Chunk 2
Chunk 3
```

Step 3

Generate embeddings.

Model:

```
nomic-embed-text
```

via Ollama.

Step 4

Store vectors.

Database:

```
ChromaDB
```

---

5. Query Pipeline

User question:

```
What support plans do you offer?
```

Process:

Step 1

Question embedding generated.

Step 2

Vector search executed.

Step 3

Top matching chunks retrieved.

Example:

```
Chunk A
Chunk B
Chunk C
```

Step 4

Context injected into prompt.

Prompt:

```
Context:
...

Question:
...
```

Step 5

Prompt sent to Ollama.

Step 6

Final answer returned.

---

6. Large Language Model Layer

Inference engine:

```
Ollama
```

Responsibilities:

- Prompt execution
- Answer generation
- Local inference

Supported models:

Examples:

- Gemma

- Qwen
- Llama
- Mistral

Current architecture supports model swapping with minimal code changes.

---

7. Speech-to-Text Layer

Technology:

```
Faster Whisper
```

Model:

```
medium
```

Responsibilities:

- Audio transcription
- Voice query conversion

Pipeline:

```
Microphone
   ↓
Audio Upload
   ↓
Faster Whisper
   ↓
Text Query
```

---

8. Text-to-Speech Layer

Technology:

```
Kokoro TTS
```

Responsibilities:

- Convert AI response to speech
- Produce natural voice output

Pipeline:

```
AI Response
    ↓
Kokoro
    ↓
WAV Audio
    ↓
Frontend Playback
```

---

9. Frontend Architecture

Technology:

- HTML
- CSS
- JavaScript

No frontend framework required.

Layout Structure

```
Header

┌──────────────────────────┐
│ Avatar  │ Visualizer     │
└──────────────────────────┘

Conversation Area

Message Composer
```

---

10. Avatar System

Purpose:

Provide conversational presence.

Implementation:

Vector SVG assistant.

Idle State:

```
Neutral mouth
```

Speaking State:

```
8-frame mouth animation
```

Animation occurs only while TTS audio is playing.

---

11. Audio Visualizer

Technology:

```
Canvas API
Web Audio API
AnalyserNode
```

Pipeline:

```
Audio Playback
      ↓
AnalyserNode
      ↓
Frequency Data
      ↓
Canvas Rendering
```

Behavior:

- Blank when idle
- Active during speech

---

12. Voice Interaction Flow

User presses microphone.

```
Microphone
    ↓
Audio Recording
    ↓
Whisper
    ↓
Text
    ↓
RAG
    ↓
Ollama
    ↓
Response
    ↓
Kokoro
    ↓
Audio Playback
```

---

13. Current Directory Structure

```
backend/

main.py
rag.py
ingest.py
stt.py
tts.py

knowledge/
└── knowledge.txt
```

```
db/
└── chroma

frontend/

index.html
styles.css
script.js
```

---

14. Future Enhancements

Communication Channels

- WhatsApp
- Telegram
- Discord
- Slack

Knowledge Sources

- Website crawling
- CRM integration
- Database retrieval

Agent Features

- Multi-agent workflows
- Tool calling
- Scheduling
- Email generation

Realtime Features

- Streaming responses
- Streaming TTS
- Realtime avatar animation
- WebSocket communication

---

15. Design Principles

The platform is designed around:

- Local-first AI
- Privacy
- Modularity
- Low operational cost
- Replaceable components
- Self-hosted deployment

Every major subsystem can be independently upgraded without redesigning the entire platform.